Paco Sun

778-697-7210 | SunweichenP@Gmail.com | https://github.com/bayareahomelander | Vancouver, Canada

Work Experience

Machine Learning Engineer June 2022 – February 2025

Z.AI *Remote, Canada*

- Fine-tuned GLM-family LLMs with **LoRA** and **4-bit QLoRA**, improving multilingual benchmark accuracy by 8% while reducing adapter training cost by roughly 30% versus full fine-tuning.
- Optimized LLM inference pipelines with **TensorRT-LLM**, **Flash Attention**, **KV-cache reuse**, and **continuous batching**, reducing median response latency by 28% under multi-user serving loads.
- Built automated retraining and evaluation workflows with **Docker**, **Kubernetes**, **MLflow**, and **TensorBoard**, cutting model iteration time from multiple days to under 24 hours for routine experiments.
- Prototyped **retrieval-augmented generation (RAG)** workflows with **vector search** and reranking, improving factual answer accuracy by 7% on internal evaluation sets.
- Collaborated with engineering and research teams to profile model bottlenecks, compare quantization settings, and validate throughput/quality tradeoffs before deployment.

Data Science Research Assistant September 2019 – April 2021

University of British Columbia *Vancouver, Canada*

- Analyzed 5,000+ social media observations with **Python**, **pandas**, and **NumPy** to identify behavioral patterns across demographic groups and support two research presentations.
- Built **supervised learning models** across 200+ engineered features, improving classification precision by 10% over baseline logistic regression.

Data Scientist Intern May 2019 – September 2019

TikTok *Shanghai, China*

- Designed **SQL-based data cleaning** and **validation workflows** for recommendation analytics datasets, reducing inconsistent records by 22% before downstream analysis.
- Evaluated recommendation experiment results across 400,000+ user interactions using **A/B testing** and **confidence intervals**, identifying feature changes associated with a 9% engagement lift.

Technical Project

Onyx: Grammar-Constrained LLM Inference Engine | *Python, Rust, MLX, CUDA*

- Built Onyx, an **LLM inference engine** with **grammar-aware speculative decoding** with greedy output identical to the target alone token for token. Reached a **1.49x aggregate speedup** over the target alone on a 48-request benchmark (RTX 2080 Ti), and **2.1–2.4x** on JSON, regex, and code.
- Hand-written **CUDA-graph draft decoding** (7.5–9.8 to 4.1–4.4 ms/token), **CUDA-graph replay** in 8-token blocks that stays bitwise-identical to one-token decoding when FP16 near-ties force recovery, and a deferred draft prefill that cut speculation's added time-to-first-token from 637 ms to 0.3 ms across the benchmark.
- Enforced regex and JSON Schema constraints per token with a **Rust engine** (DFAs and a stack-based JSON state machine, exposed via **PyO3**), applied as cached GPU masks; caching vocabulary scans by grammar state lifted long constrained outputs **from 0.49–0.99x to as high as 2.0x**.
- Served through an **OpenAI-style FastAPI API** with SSE streaming, with output parity enforced by real-model GPU tests; a separate **Apple Silicon** (MLX, 4-bit) implementation reached **1.45x** (15.6 to 22.6 tok/s) with an 8B target.

Education

Bachelor's Degree Vancouver, Canada

University of British Columbia *2016 - 2021*

Skills

Machine Learning: PyTorch, TensorFlow, Scikit-Learn, Transformers, LoRA/QLoRA, RAG, TensorRT-LLM, Vector Database
Inference Systems: MLX, Flash Attention, Continuous Batching, Quantization, KV Cache Optimization, Apple Silicon
Data Analytics: pandas, NumPy, SQL
Programming Languages: Python, Rust
Deployment: Docker, Kubernetes, FastAPI, MLflow